**Performance evaluation of a multi-tier business application**

Your task is to investigate how user populations, dispatching decisions and finite capacities affect a multi-tier application.

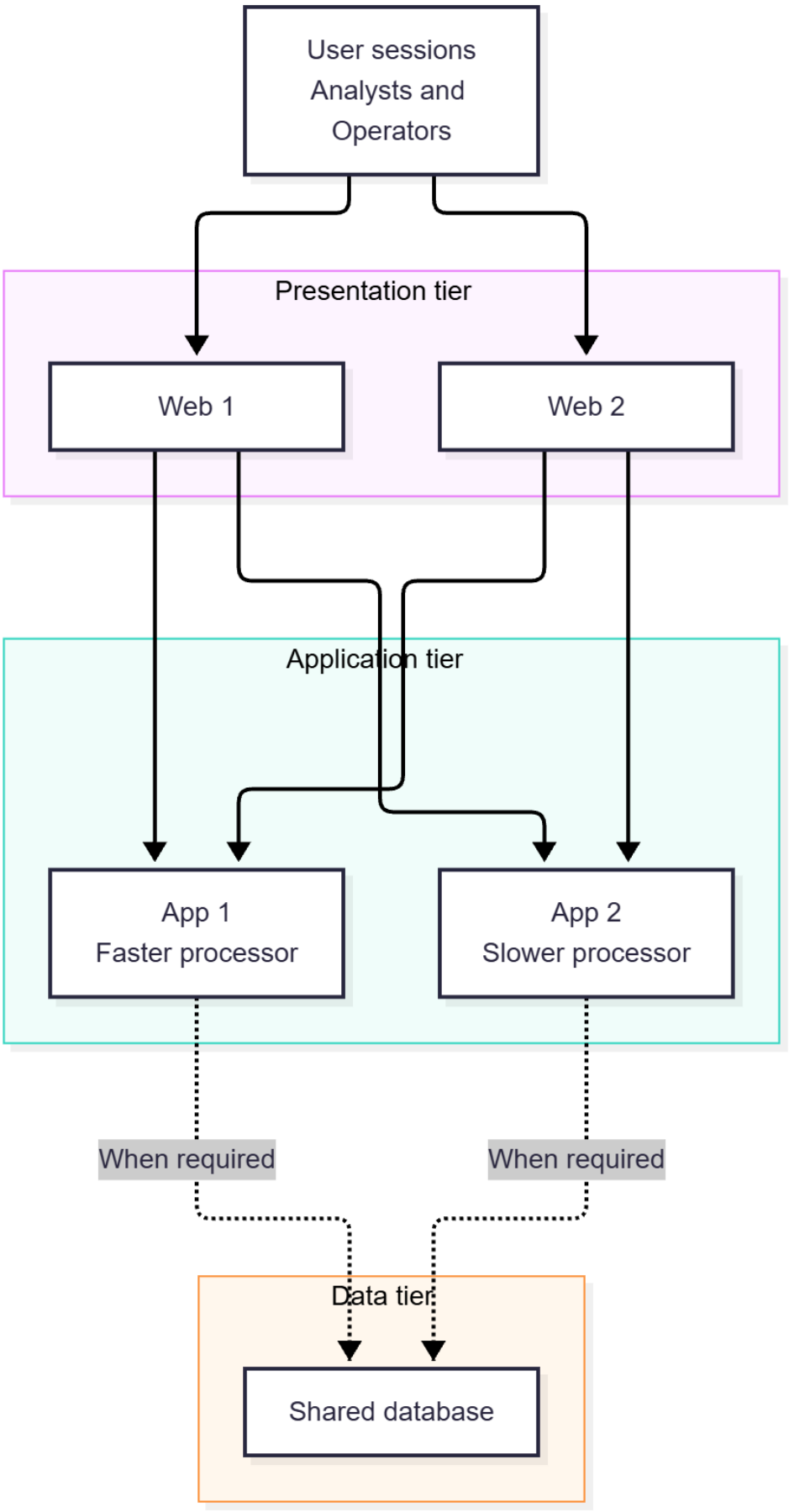
**1. System architecture**

The application supports two user groups:

- **Analysts (A)** prepare reports using in-memory business data. Their requests require substantial application processing and occasionally access the database.
- **Operators (O)** perform short application operations that usually require a database transaction.

The infrastructure consists of two Web servers, two application servers and one database server. App 1 has twice the processing speed of App 2.

The diagram shows the application architecture. It is not a queueing-network model. Return traffic, user think times and blocking behavior are omitted.




Connections represent access or service dependencies. Response traffic is omitted.

**2. Users and transaction lifecycle**

A closed-loop load test represents 24 concurrent user sessions. Each session belongs permanently to one class.

A user submits one transaction, waits for its completion, then thinks before submitting the next transaction. Each user has at most one outstanding transaction.

Think times are independent and exponentially distributed, with mean 0.50 s for both classes. Think times overlap freely and consume no application resources.

The baseline population is:

| Class | Number of users |
|---|---|
| **Analysts** | 22 |
| **Operators** | 2 |

There are no external arrivals, class changes, timeouts, retries or abandoned transactions.

Each transaction follows these rules:

Web assignment. Successive transactions of each class are assigned cyclically to Web 1 and Web 2. The assignment sequence is maintained separately for each class.

Application assignment. After Web processing, the Web server selects the application server with the smallest current number of requests, counting queued requests and requests in service. Use JMT's built-in tie handling.

Database access. After application processing, a transaction requires exactly one database operation with probability 0.20 for Analysts and 0.90 for Operators. Otherwise, it completes immediately.

A database operation completes the transaction. There is no additional Web or application processing.

Database-access decisions are independent across transactions and do not change the user's class.

Dispatching has negligible processing cost. It is part of the sending component's routing behavior and provides no additional buffering.

### 3. Processing resources

All service requirements are independent. The table gives the mean processing time and coefficient of variation per visit.

| Resource | Processing units | Capacity $K$ | Analyst service time ($\mu$) and variation ($c_v$) | Operator service time ($\mu$) and variation ($c_v$) |
|---|---|---|---|---|
| **Web 1** | 1 | 4 | $\mu =$ 2 ms, $c_v = 0.5$ | $\mu =$ 2 ms, $c_v = 1.5$ |
| **Web 2** | 1 | 4 | $\mu =$ 2 ms, $c_v = 0.5$ | $\mu =$ 2 ms, $c_v = 1.5$ |
| **App 1** | 1 | 8 | $\mu =$ 80 ms, $c_v = 1.0$ | $\mu =$ 8 ms, $c_v = 1.0$ |
| **App 2** | 1 | 8 | $\mu =$ 160 ms, $c_v = 1.0$ | $\mu =$ 16 ms, $c_v = 1.0$ |
| **Database** | 1 | 4 | $\mu =$ 10 ms, $c_v = 2.0$ | $\mu =$ 80 ms, $c_v = 0.25$ |
| **Think time** | | | Z = 500 ms, $c_v = 1.0$ | Z = 500 ms, $c_v = 1.0$ |

The resources operate as follows:

Each Web server processes one request at a time, in arrival order.

Each application server divides its processor equally among all requests currently receiving processing, regardless of class. Switching overhead is negligible.

The database processes one operation at a time, in arrival order.

There are no class priorities.

The capacity $K$ is shared by both classes. Every admitted request consumes one slot. It is a station-capacity limit, not a number of processors.

When a destination is full, transfer is blocked after upstream processing has completed. The completed processing is not repeated, and the request is not discarded or reassigned to another destination. Use JMT's BAS policy for both classes at every finite-capacity station.

Use JMT's native BAS semantics. Do not insert additional unlimited queues between processing stages. User think-time facilities have no capacity limit.

All other communication and processing costs are negligible.

### 4. Performance definitions

Let:

- $X_A$, $X_O$ denote completed transactions per second for each class.
- $R_A$, $R_O$ denote mean transaction response times, from submission to completion, including blocking and queueing but excluding think time.
- $U_i$ denote the utilization of processing resource $i$.

**5. Tasks**

**A. Construct the model**

Build the baseline model in *JSIMgraph*.

Describe:

- The workload representation.
- The scheduling discipline and capacity of each station.
- Where each of the three routing strategies is applied.
- What happens when a destination is full.

Explain why the two user classes cannot be analyzed as independent single-class networks sharing no resources.

**B. Establish and validate the baseline**

Run the baseline configuration and report:

- $X_A$, $X_O$ and total transaction throughput $X$.
- $R_A$, $R_O$ and average system response time R.
- The utilization of every processing resource.

Use 95% confidence intervals and target a maximum relative error of 5% for class throughputs, with 10M samples. Report whether the target was achieved.

Identify the limiting processing resource or tier (a tier is a set of resources offering the same functionality). Support your answer with numerical evidence.

**C. Study the population mix**

Keep the total population fixed at 24 and evaluate these three mixes:

| Configuration | Analysts | Operators |
|---|---|---|
| M1 — Baseline | 22 | 2 |
| M2 — Balanced population | 12 | 12 |
| M3 — Operator-heavy population | 2 | 22 |

Only the populations change. Please note that M1 is the case considered in Section B.

Use your results to answer:

1. Bottleneck switch. Does the limiting resource or tier change across the tested mixes? Explain the mechanism.
2. Aggregate performance. Which tested mix achieves the highest total throughput

**D. Evaluate two proposed changes**

Evaluate each proposal against its specified reference configuration. Do not combine the changes.

| Proposal | Reference configuration | Change |
|---|---|---|
| D1 — Cyclic application assignment | M1 | Replace application-tier JSQ with round robin at both Web servers, separately for each class |
| D2 — Larger database capacity | M3 | Increase database capacity from 4 to 12; keep its processing speed and scheduling unchanged |

For D1:

- Compare total throughput, class response times and application-server utilizations.
- Evaluate the claim: “Sending equal numbers of requests to both application servers, balances their load.”
- Explain why JSQ is not equivalent to choosing the server with the smallest expected completion time.

For D2:

- Compare Operator throughput and both class response times.
- Evaluate the claim: “Tripling database capacity should increase Operator throughput by at least 20%.”
- Explain the distinction between increasing storage for waiting requests and increasing processing capacity.

Do not interpret small differences as improvements when your simulation uncertainty does not support that conclusion.

**5. Submission**

Submit:

- The native JMT model for each of the four configurations (initial, new workload, alternative S and alternative V).
- One compact results table covering all 5 configurations (M1, M2, M3, D1 and D2), creating using any tool of your choice (i.e. Word, Excel, PowerPoint, Latex...) in PDF. If you are in a hurry, in the table you can avoid reporting the utilizations.

You may consult your notes, search the web and use AI tools. You are responsible for checking all assumptions, model settings and conclusions. Answers must refer to your submitted JMT model and numerical results.

Enter your answers in the following form:

Performance Evaluation and Applications 2026-2027 - In Class Test 2 – Compila modulo

https://forms.cloud.microsoft/Pages/ResponsePage.aspx?id=K3EXCvNtXUKAjjCd8ope67-7CBR7gDJEgHF_krAEqPhUOE1KRUQ5QktBUUMxV1JOSURWOE9aUldGWi4u